Bayesian Learning - Computer Lab 3

Liuxi Mei Xiaochen Liu

2025-0-13

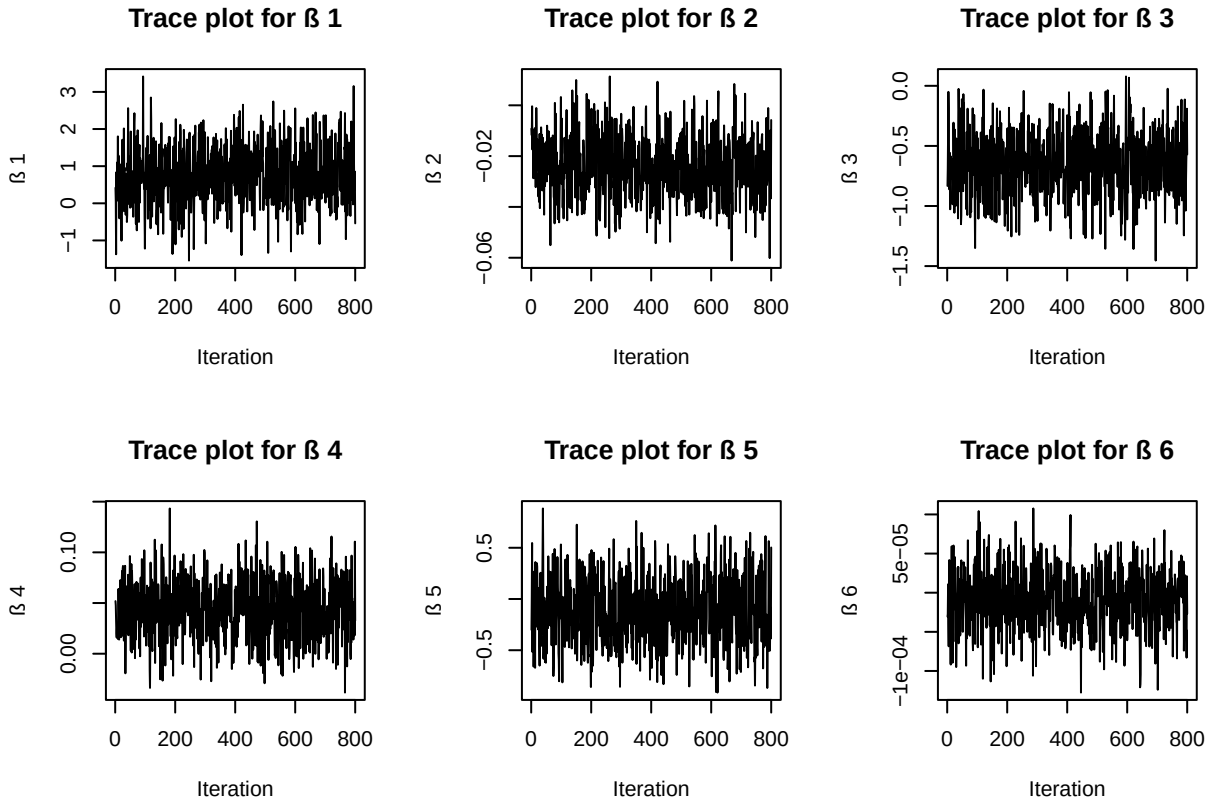
1. Gibbs sampling for the logistic regression

Consider again the logistic regression model in problem 2 from the previous computer lab 2. Use the prior $\beta \sim \mathcal{N}(0, \tau^2 I)$, where $\tau = 3$.

(a) Implement (code!) a Gibbs sampler that simulates from the joint posterior $p(\omega, \beta|x)$ by augmenting the data with Polya-gamma latent variables $\omega_i, i = 1, \ldots, n$. The full conditional posteriors are given on the slides from Lecture 7. Evaluate the convergence of the Gibbs sampler by calculating the Inefficiency Factors (IFs) and by plotting the trajectories of the sampled Markov chains.

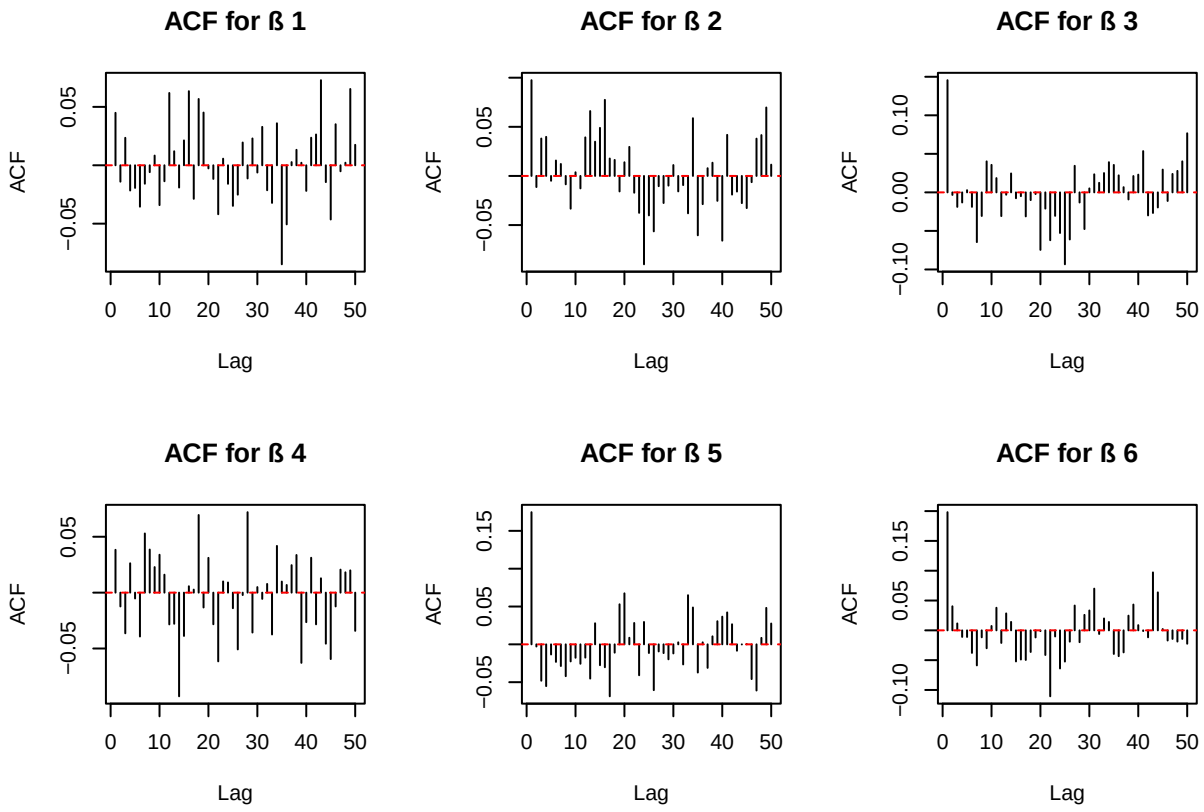
```
library(BayesLogit)
library(mvtnorm)

data = read.csv('Disease.csv')
x <- as.matrix(data [, 1:6])
y <- as.matrix(data [, 7])
# initialize beta
nDraws = 1000
burnin <- 200       # Burn-in period

n <- nrow(x)
p <- ncol(x)

beta <- rep(1, 6) # 5+1 intercept

beta_samples <- matrix(NA, nDraws, p)

set.seed(123)

for (i in 1:nDraws) {
  omega <- rpg(n, 1, x %*% beta)
  k <- y - 1 / 2
  B <- 9 * diag(6)
  b <- rep(0, p)
  # Conditional posterior for beta

  v_omg <- solve(t(x) %*% diag(omega) %*% x + solve(B))
  m_omg <- v_omg %*% (t(x) %*% k + solve(B) %*% b)

  beta <- as.numeric(rmvnorm(1, mean = m_omg, sigma = v_omg))
  beta_samples[i, ] <- beta
```

```
}
# Discard burn-in
beta_samples <- beta_samples[(burnin + 1):nDraws, ]
nPost <- nrow(beta_samples)

par(mfrow = c(2, 3))  # 6 pics

for (i in 1:p) {
  plot(
    beta_samples[, i],
    type = 'l',
    main = paste("Trace plot for  ", i),
    xlab = "Iteration",
    ylab = paste(" ", i)
  )
}
```




```
max_lag <- 50  # set the max lag
acfs <- matrix(NA, max_lag, p)

for (i in 1:p) {
  chain <- beta_samples[, i] - mean(beta_samples[, i])
  var_chain <- mean(chain ^ 2)
  for (j in 1:max_lag) {  # j as lag
    # calculate the autocorrelation factor
```

```r
    acfs[j, i] <- sum(chain[1:(nPost - j)] * chain[(j + 1):nPost]) / nPost / var_chain

  }

}

par(mfrow = c(2, 3))  # 6 pics

for (i in 1:p) {
  plot(
    1:max_lag,
    acfs[, i],
    type = 'h',
    main = paste("ACF for  ", i),
    xlab = "Lag",
    ylab = "ACF"
  )
  abline(h = 0, col = "red", lty = 2)
}
```




```r
# inefficiency factor (IF)
IFs <- numeric(p)

for (i in 1:p) {
  IFs[i] <- 1 + 2 * sum(acfs[, i])
```

```
  cat(paste("IF for  ", i, "=", round(IFs[i], 2), "\n"))
}
```

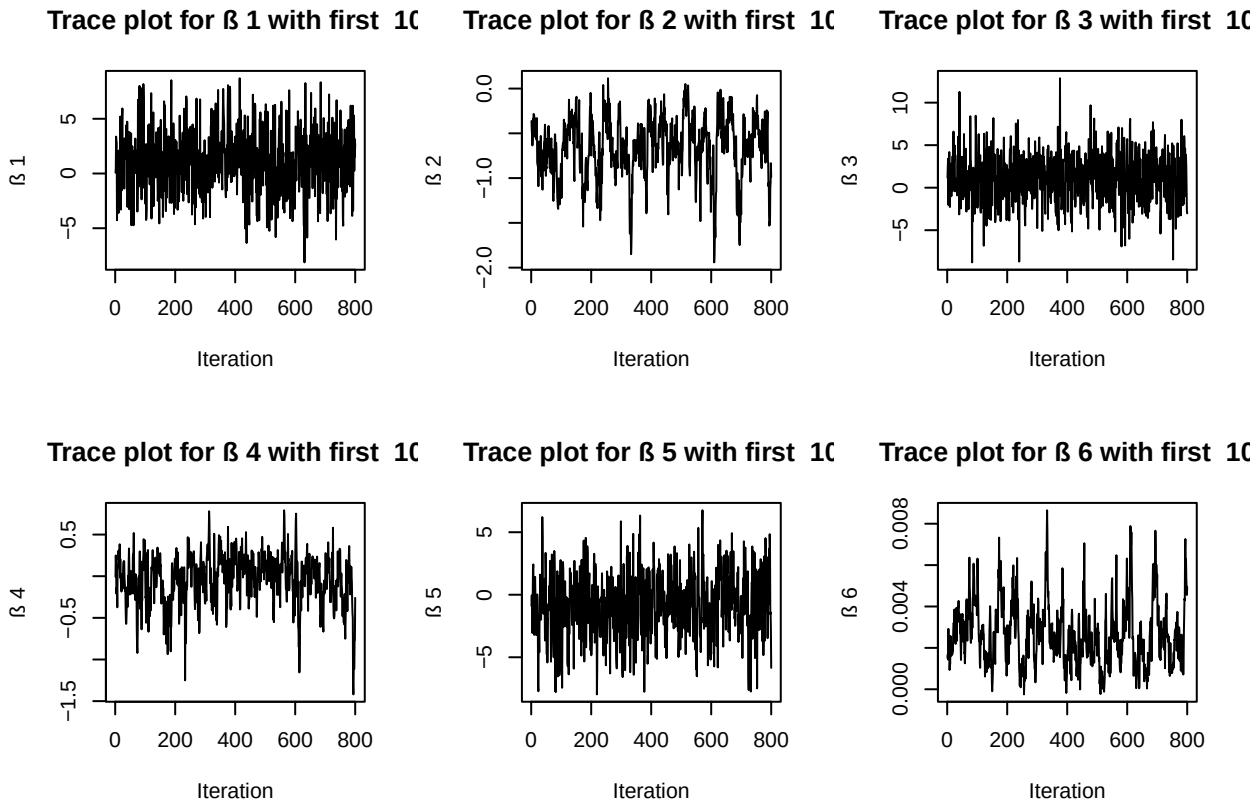
```
## IF for   1 = 1.16
## IF for   2 = 1.25
## IF for   3 = 0.95
## IF for   4 = 0.72
## IF for   5 = 0.77
## IF for   6 = 0.72
```

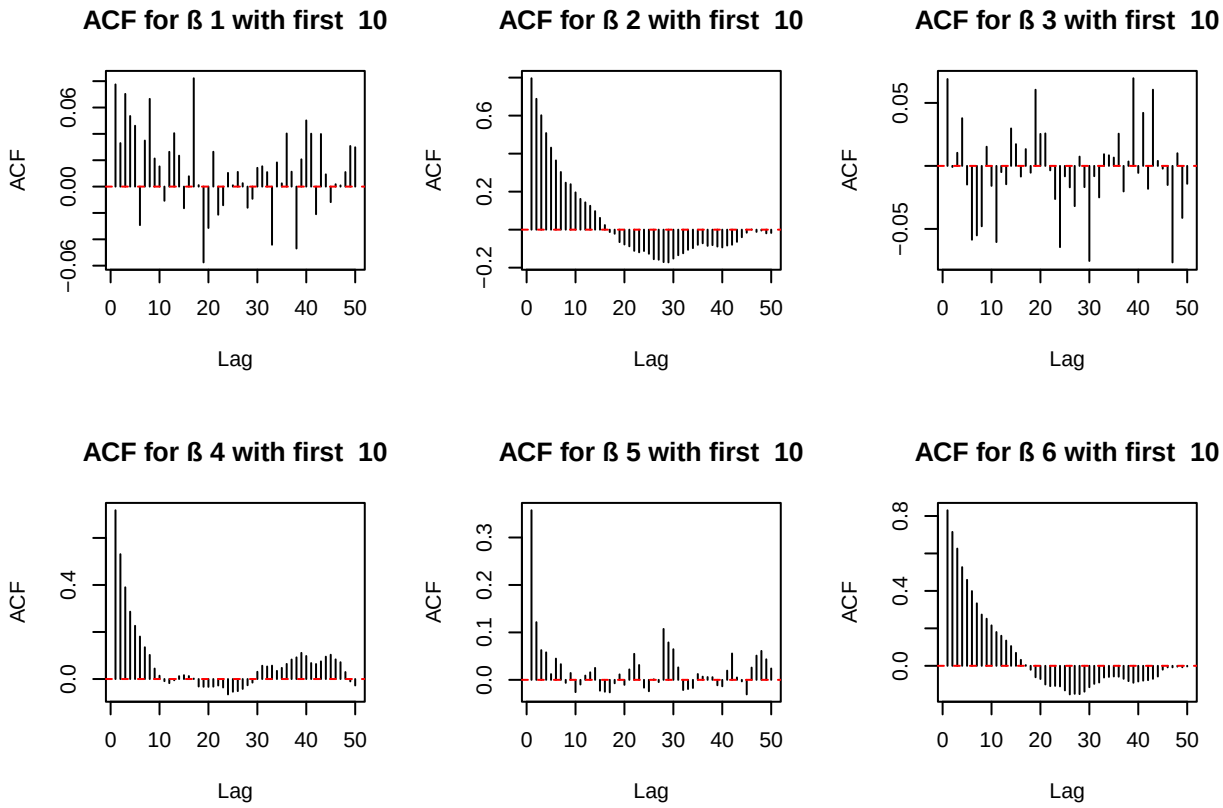
Plots show that all efficient fluctrate around certain levels without any obvious tendency or any stuck. And calculated inefficiency factors are around 1, which means very sampling are performed with quite low dependency and the efficiency is very good.

(b) Repeat the same task as in (a), but now only use the first m observations of the dataset. Do this for all $m \in \{10, 40, 80\}$.

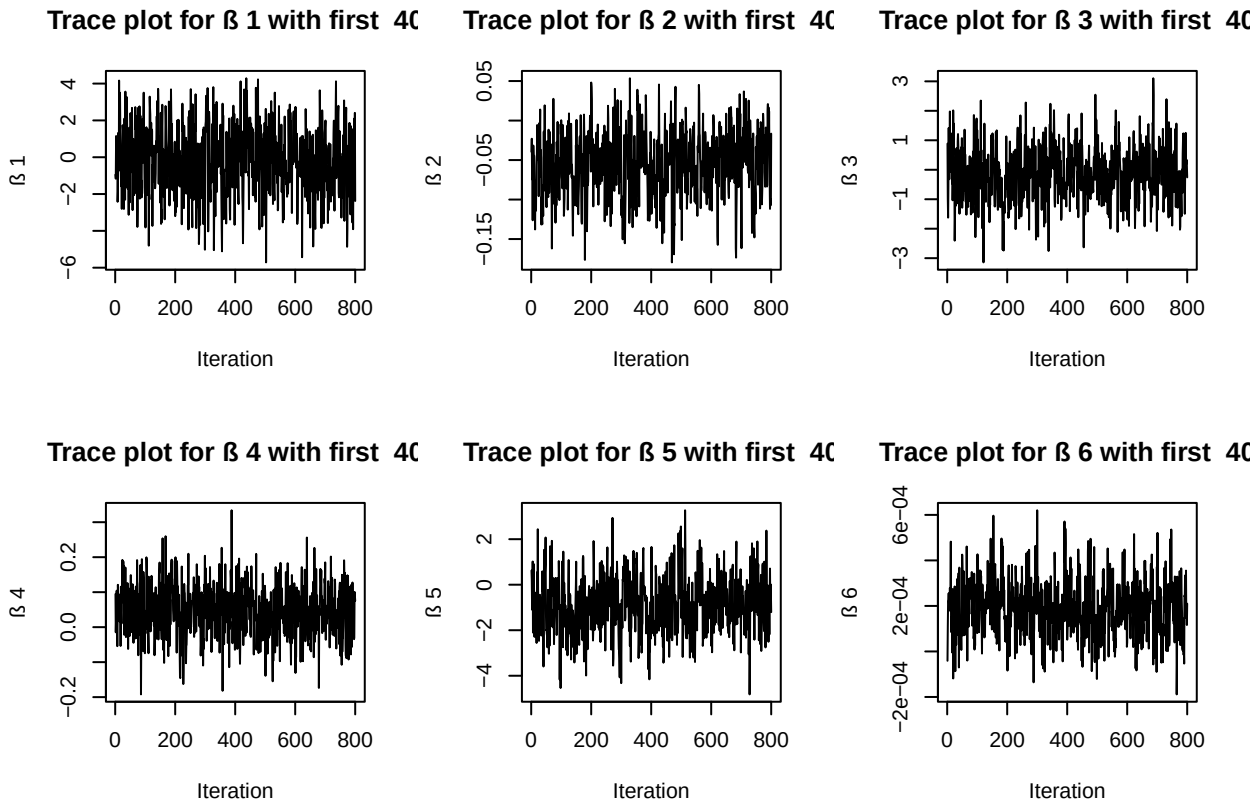
```
m <- c(10, 40, 80)
for (z in 1:3) {
  row_nr <- m[z]
  data = read.csv('Disease.csv')
  x <- as.matrix(data [1:row_nr, 1:6])
  y <- as.matrix(data [1:row_nr, 7])

  # initialize beta
nDraws = 1000
burnin <- 200       # Burn-in period

n <- nrow(x)
p <- ncol(x)

beta <- rep(1, 6) # 5+1 intercept

beta_samples <- matrix(NA, nDraws, p)

set.seed(123)

for (i in 1:nDraws) {
  omega <- rpg(n, 1, x %*% beta)
  k <- y - 1 / 2
  B <- 9 * diag(6)
  b <- rep(0, p)
  # Conditional posterior for beta

  v_omg <- solve(t(x) %*% diag(omega) %*% x + solve(B))
  m_omg <- v_omg %*% (t(x) %*% k + solve(B) %*% b)

  beta <- as.numeric(rmvnorm(1, mean = m_omg, sigma = v_omg))
  beta_samples[i, ] <- beta
}
# Discard burn-in
beta_samples <- beta_samples[(burnin + 1):nDraws, ]
```

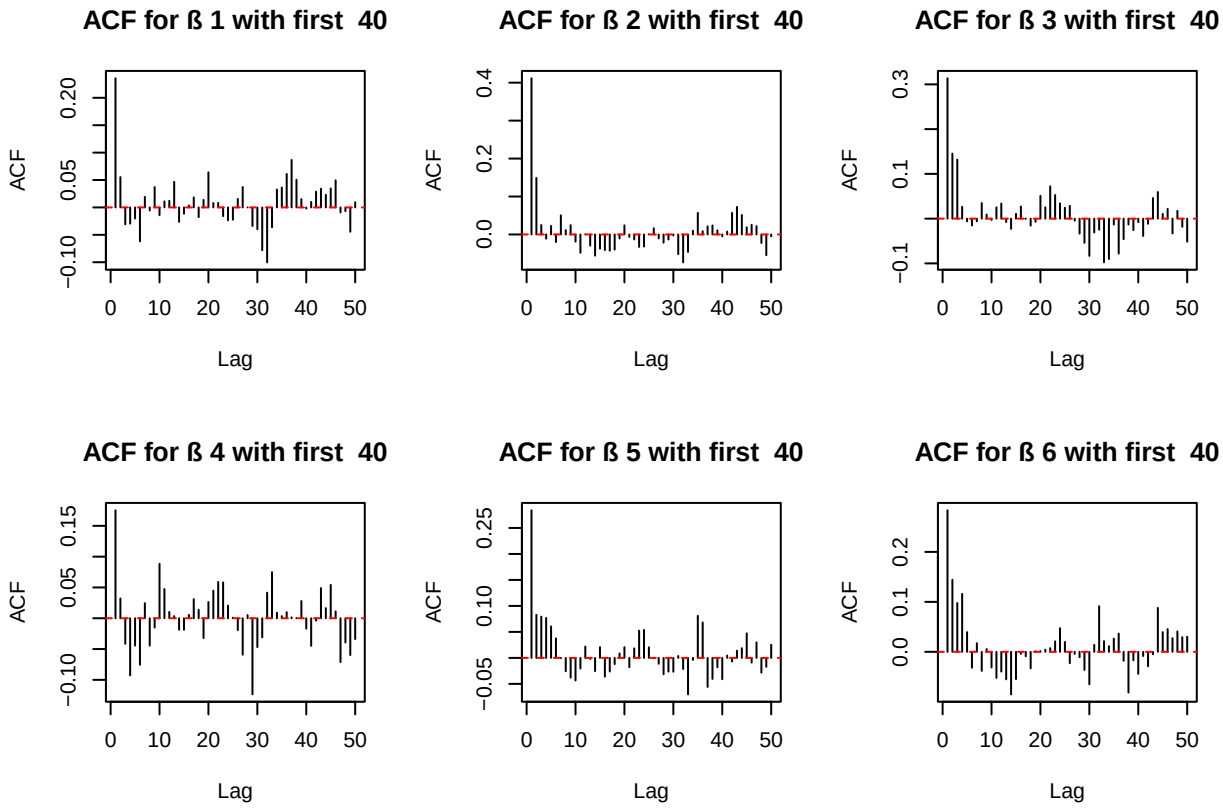
```r
nPost <- nrow(beta_samples)

par(mfrow = c(2, 3))  # 6 pics

for (i in 1:p) {
  plot(
    beta_samples[, i],
    type = 'l',
    main = paste("Trace plot for  ", i , "with first ", row_nr),
    xlab = "Iteration",
    ylab = paste(" ", i)
  )
}


max_lag <- 50  # set the max lag
acfs <- matrix(NA, max_lag, p)

for (i in 1:p) {
  chain <- beta_samples[, i] - mean(beta_samples[, i])
  var_chain <- mean(chain ^ 2)
  for (j in 1:max_lag) {  # j as lag
    # calculate the autocorrelation factor
    acfs[j, i] <- sum(chain[1:(nPost - j)] * chain[(j + 1):nPost]) / nPost / var_chain

  }

}

par(mfrow = c(2, 3))  # 6 pics

for (i in 1:p) {
  plot(
    1:max_lag,
    acfs[, i],
    type = 'h',
    main = paste("ACF for  ", i , "with first ", row_nr),
    xlab = "Lag",
    ylab = "ACF"
  )
  abline(h = 0, col = "red", lty = 2)
}

# inefficiency factor (IF)
IFs <- numeric(p)

for (i in 1:p) {
  IFs[i] <- 1 + 2 * sum(acfs[, i])
  cat(paste("IF for  ", i, "=", round(IFs[i], 2), "with first ", row_nr, "\n"))
}

}
```


Trace plot for ß 1 with first 10
ß 1
Iteration
Trace plot for ß 2 with first 10
ß 2
Iteration
Trace plot for ß 3 with first 10
ß 3
Iteration
Trace plot for ß 4 with first 10
ß 4
Iteration
Trace plot for ß 5 with first 10
ß 5
Iteration
Trace plot for ß 6 with first 10
ß 6
Iteration

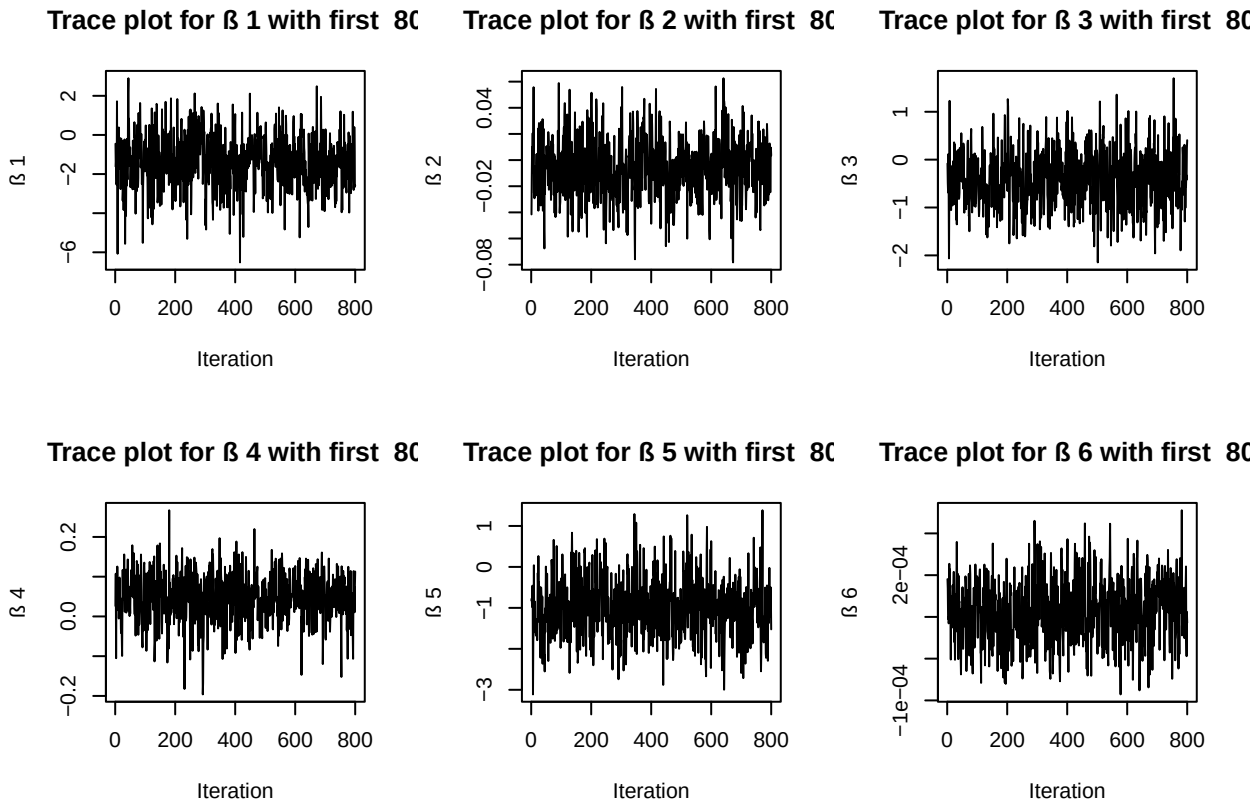


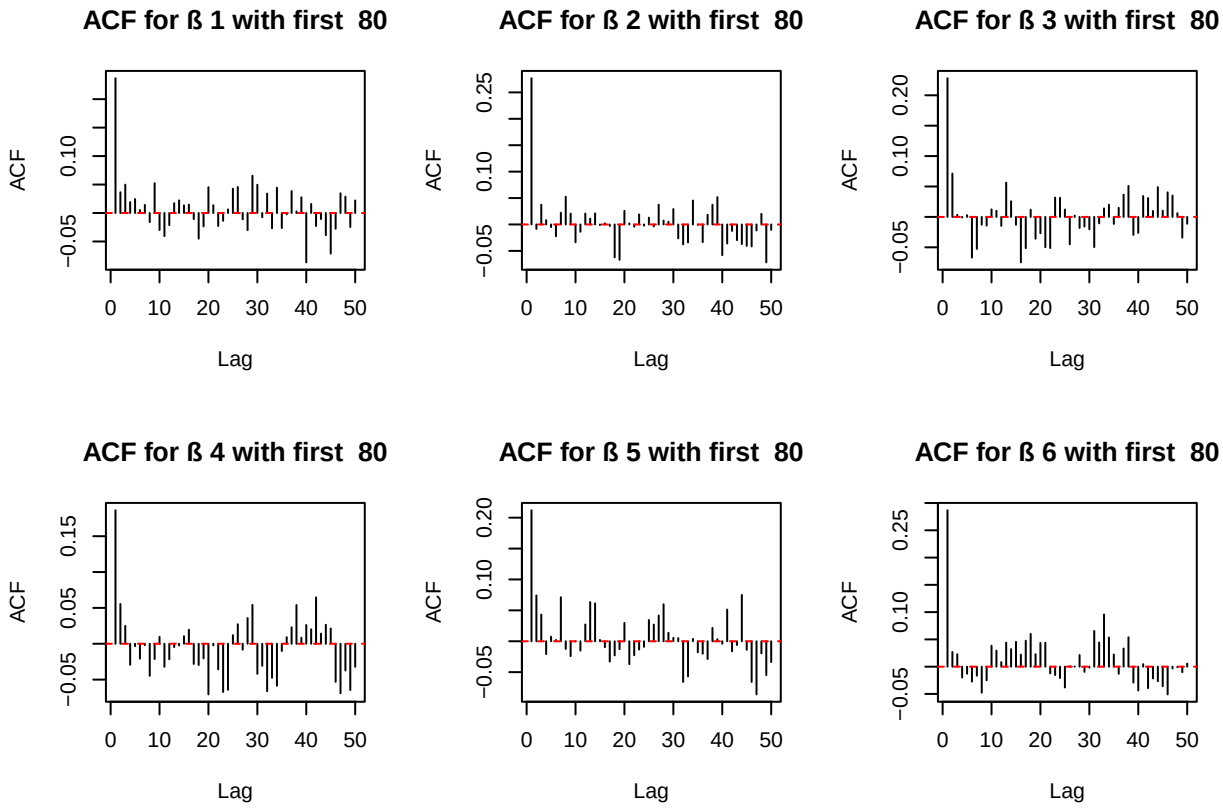

```
## IF for   1 = 2.34 with first  10
## IF for   2 = 5.3 with first  10
## IF for   3 = 0.59 with first  10
## IF for   4 = 7.94 with first  10
## IF for   5 = 3.34 with first  10
## IF for   6 = 6.65 with first  10
```


Trace plot for ß 1 with first 40
ß 1
−6
−2
0
2
4
0
200
400
600
800
Iteration
Trace plot for ß 2 with first 40
ß 2
−0.15
−0.05
0.05
0
200
400
600
800
Iteration
Trace plot for ß 3 with first 40
ß 3
−3
−1
1
3
0
200
400
600
800
Iteration
Trace plot for ß 4 with first 40
ß 4
−0.2
0.0
0.2
0
200
400
600
800
Iteration
Trace plot for ß 5 with first 40
ß 5
−4
−2
0
2
0
200
400
600
800
Iteration
Trace plot for ß 6 with first 40
ß 6
−2e−04
2e−04
6e−04
0
200
400
600
800
Iteration




```
## IF for   1 = 1.85 with first  40
## IF for   2 = 1.75 with first  40
## IF for   3 = 1.73 with first  40
## IF for   4 = 1.02 with first  40
## IF for   5 = 1.95 with first  40
## IF for   6 = 2.06 with first  40
```


Trace plot for ß 1 with first 80
ß 1
−6
−2
0
2
0
200
400
600
800
Iteration
Trace plot for ß 2 with first 80
ß 2
−0.08
−0.02
0.04
0
200
400
600
800
Iteration
Trace plot for ß 3 with first 80
ß 3
−2
−1
0
1
0
200
400
600
800
Iteration
Trace plot for ß 4 with first 80
ß 4
−0.2
0.0
0.2
0
200
400
600
800
Iteration
Trace plot for ß 5 with first 80
ß 5
−3
−1
0
1
0
200
400
600
800
Iteration
Trace plot for ß 6 with first 80
ß 6
−1e−04
2e−04
0
200
400
600
800
Iteration




```
## IF for   1 = 1.83 with first  80
## IF for   2 = 1.15 with first  80
## IF for   3 = 1.23 with first  80
## IF for   4 = 0.36 with first  80
## IF for   5 = 1.42 with first  80
## IF for   6 = 2.31 with first  80
```

As for case 1 where we have 10 observations, there is a decreasing tendency noticed for $\beta 4$ and the overall IF is comparatively high, thus it is not considered to converge.

For case 2 and 3 where there are 40 and 80 observations respectively, all plots have reached a stabilization and all parameters have a good IF factor. Thus convergence is thought to be reached.

2. Metropolis Random Walk for Poisson regression

Consider the following Poisson regression model:

$$y_i|\beta \sim \text{Poisson}\left(\exp(x_i^T\beta)\right), \quad i = 1, \ldots, n,$$

where y_i is the count for the ith observation in the sample and x_i is the p-dimensional vector with covariate observations for the ith observation.

The dataset contains observations from 700 eBay auctions of coins. The response variable is `nBids` and records the number of bids in each auction. The remaining variables are features/covariates (x):

- **Const**: Intercept
- **PowerSeller**: Equal to 1 if the seller is selling large volumes on eBay
- **VerifyID**: Equal to 1 if the seller is a verified seller by eBay
- **Sealed**: Equal to 1 if the coin was sold in an unopened envelope
- **MinBlem**: Equal to 1 if the coin has a minor defect
- **MajBlem**: Equal to 1 if the coin has a major defect
- **LargNeg**: Equal to 1 if the seller received a lot of negative feedback from customers
- **LogBook**: Logarithm of the book value of the auctioned coin according to expert sellers (Standardized)
- **MinBidShare**: Ratio of the minimum selling price (starting price) to the book value (Standardized).

(a)

Obtain the maximum likelihood estimator of β in the Poisson regression model for the eBay data. [Hint: `glm.R`, don't forget that `glm()` adds its own intercept so don't input the covariate `Const`.] Which covariates are significant?

```
rm(list=ls())

data <-read.table('eBayNumberOfBidderData_2025.dat')
y<- matrix(as.numeric(data$V1), ncol=1)
```

```
## Warning in matrix(as.numeric(data$V1), ncol = 1):      NA
```

```
x<- as.matrix(data[, 2:10])
x[,8:9] <- as.numeric(x[,8:9])
```

```
## Warning:      NA
```

(b)

Let's do a Bayesian analysis of the Poisson regression. Let the prior be $\beta \sim \mathcal{N}(0, 100 \cdot (X^\top X)^{-1})$, where X is the $n \times p$ covariate matrix. This is a commonly used prior, which is called Zellner's g-prior. Assume first that the posterior density is approximately multivariate normal:

$$\beta \mid y \sim \mathcal{N}(\tilde{\beta}, J_y^{-1}(\tilde{\beta})),$$

where $\tilde{\beta}$ is the posterior mode and $J_y(\tilde{\beta})$ is the negative Hessian at the posterior mode.

$\tilde{\beta}$ and $J_y(\tilde{\beta})$ can be obtained by numerical optimization (`optim.R`) exactly like you already did for the logistic regression in Lab 2 (but with the log posterior function replaced by the corresponding one for the Poisson model, which you have to code up).

(c)

Let's simulate from the actual posterior of β using the Metropolis algorithm and compare the results with the approximate results in (b). Program a general function that uses the Metropolis algorithm to generate random draws from an arbitrary posterior density. In order to show that it is a general function for any model, we denote the vector of model parameters by θ. Let the proposal density be the multivariate normal density mentioned in Lecture 8 (random walk Metropolis):

$$\theta^{(p)} \mid \theta^{(i-1)} \sim \mathcal{N}(\theta^{(i-1)}, c \cdot \Sigma),$$

where $\Sigma = J_y^{-1}(\tilde{\beta})$ was obtained in (b). The value c is a tuning parameter and should be an input to your Metropolis function. The user of your Metropolis function should be able to supply her own posterior density function, not necessarily for the Poisson regression, and still be able to use your Metropolis function. This is not so straightforward, unless you have come across function objects in R. The note `HowToCodeRWM.pdf` in Lisam describes how you can do this in R.

Now, use your new Metropolis function to sample from the posterior of β in the Poisson regression for the eBay dataset. Assess MCMC convergence by graphical methods.

(d)

Use the MCMC draws from (c) to simulate from the predictive distribution of the number of bidders in a new auction with the characteristics below. Plot the predictive distribution. What is the probability of no bidders in this new auction?

- PowerSeller = 1
- VerifyID = 0
- Sealed = 1
- MinBlem = 0
- MajBlem = 1
- LargNeg = 0
- LogBook = 1.3
- MinBidShare = 0.7

3.Time series models in Stan

(a) Simulating an AR(1) Process

Write a function in R that simulates data from the AR(1) process:

$$x_t = mu + phi(x_{t-1} - mu) + varepsilon_t, quadvarepsilon_t oversettextiidsimmathcalN(0, sigma^2),$$

for given values of
mu,
phi, and
$sigma^2$. Start the process at $x_1 =$
mu and then simulate values for x_t for $t = 2, 3,$
$dots, T$, and return the vector $x_{1:T}$ containing all time points.

Use
$mu = 5$,

$sigma^2 = 9$, and $T = 300$, and look at some different realizations (simulations) of $x_{1:T}$ for values of phi between -1 and 1 (this is the interval of
phi where the AR(1) process is stationary). Include a plot of at least one realization in the report.

**What effect does the value of
phi have on $x_{1:T}$?**

(b) Bayesian Inference for AR(1) Parameters

Use your function from (a) to simulate two AR(1) processes:

- $x_{1:T}$ with
  $phi = 0.4$
- $y_{1:T}$ with
  $phi = 0.98$

Now, treat your simulated vectors as synthetic data, and treat the values of
mu,
phi, and
$sigma^2$ as unknown parameters. Implement **Stan code** that samples from the posterior of the three parameters, using suitable non-informative priors of your choice.

> **Hint:** Look at the time-series models examples in the Stan User's Guide/Reference Manual, and note the different parameterization used here.

(i) Posterior Summaries

Report the **posterior mean**, **95% credible intervals**, and the **number of effective posterior samples** for the three inferred parameters for each of the simulated AR(1) processes.
Are you able to estimate the true values?

(ii) Convergence and Joint Posterior

For each of the two data sets:

- Evaluate the **convergence** of the samplers.
- Plot the **joint posterior** of
  mu and
  phi.

Comments?